

官方发布 Claude Skills 的最佳实践指南

 mp.weixin.qq.com/s/gOl1Fg6-B7BtqlxmQW_Xcw

程序员安仔



最近这段时间，Claude Skills 在 AI 圈子里可谓是火得一塌糊涂。

从开发者社区到各大技术论坛，大家都在热火朝天地折腾各种 Skills：

有人做自动化工作流；

有人搞内容创作助手；

有人甚至把整个项目管理流程都封装成了 Skills。

眼看着社区玩得越来越花，Anthropic 官方也坐不住了，最近正式发布了一份 **Claude Skills 构建完全指南**，系统性地梳理了最佳实践、设计模式和避坑指南，算是给这股热潮提供了一份“官方教科书”。

今天我把这篇指南翻译为中文了，并在中间穿插了一些个人见解来方便大家来理解学习，希望是小白也能快速上手用起来。

好了，那 Skills 到底是什么东西呢？

简单来说，**Skills 就是一套打包好的指令，教会 Claude 如何处理你的特定工作流程。**

你不用每次对话都重新解释一遍你的需求、流程和专业知识，只需要教 Claude 一次，之后每次都能直接用。

这可能是让 Claude 变得更实用和平民化的一个重要转折点。

就举个例子，你是个产品经理，每周都要做冲刺规划。

以前你得跟 Claude 解释你们团队的流程、Linear 怎么用、任务怎么分配。

现在有了 Skills，这些知识被固化下来了，你只需要说“帮我规划这个冲刺”，Claude 就知道该做什么，该怎么做，甚至知道要调用哪些工具。

使用场景	没有 Skills	有了 Skills
每次对话	重新解释流程和需求	直接说任务目标
工具调用	手动指导每一步	自动按流程执行
结果一致性	因提示不同而变化	标准化输出
学习成本	每个用户都要学	一次配置团队共享

Skills 到底长什么样

从技术角度看，Skills 就是一个文件夹，里面最核心的是一个叫 **SKILL.md** 的文件。

这个文件用 Markdown 格式写成，顶部有 YAML 格式的元数据。

除此之外，你还可以放一些可执行脚本、参考文档、模板素材等等。

整个结构非常简洁，但威力巨大。

说白了，这就像给 Claude 准备了一本“操作手册”。

你平时怎么跟同事交接工作的？

写个文档，列清楚步骤，附上参考资料对吧？

Skills 就是这个思路，只不过这次你的“同事”是 AI。

Skills 的文件结构：

your-skill-name/

- └─ SKILL.md # 必需 - 主要的 Skill 文件
- └─ scripts/ # 可选 - 可执行代码
- └─ references/ # 可选 - 参考文档
- └─ assets/ # 可选 - 模板、字体、图标



Anthropic 在设计 Skills 时用了一个很聪明的“**渐进式披露**”系统，分三个层级：

层级	内容	加载时机	作用
第一层	YAML 元数据	永远加载在系统提示中	让 Claude 知道何时使用，不占太多上下文
第二层	SKILL.md正文	当 Claude 认为相关时	提供完整的指令和指导
第三层	链接的额外文件	按需查看	详细文档和参考资料

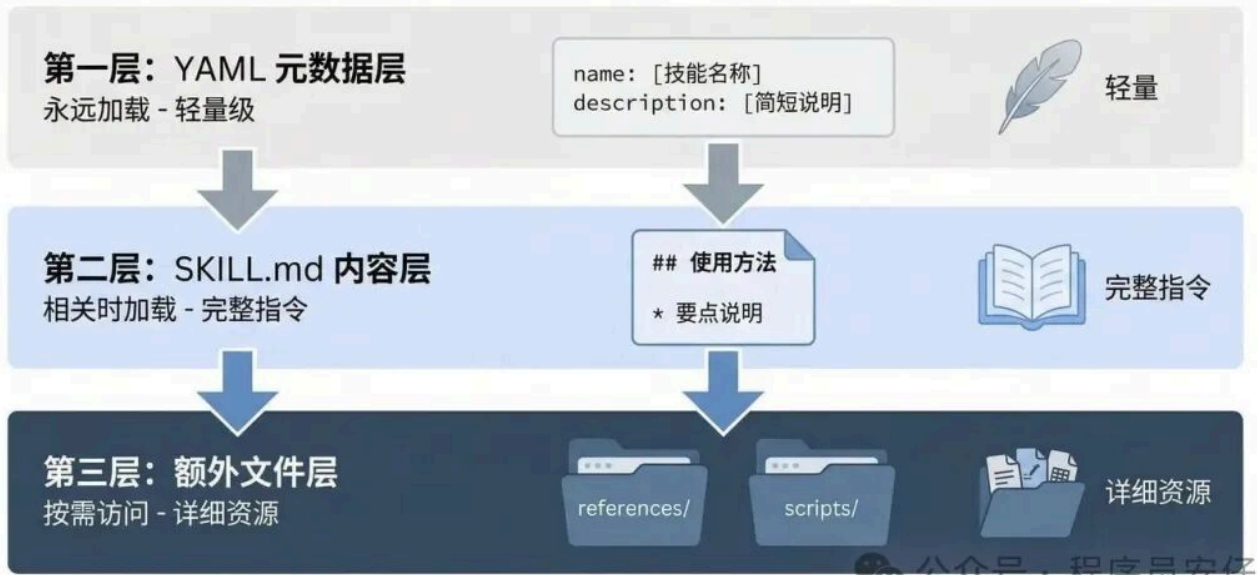
这种设计既节省了 token，适当时候又能保持专业性。

我感觉可以类比为，这有点像你手机上的 App 图标、App 界面、App 设置页面三层结构。

第一层让你知道“这是啥”，第二层让你“开始用”，第三层是“高级功能”。

Claude 不会一上来就把所有东西塞给你，而是按需加载，这样能省掉不少资源，响应速度也快（还能间接帮你省钱）。

CLAUDE SKILLS: 渐进式披露系统



更重要的是，**Skills** 具有可组合性。

Claude 可以同时加载多个 **Skills**，它们能协同工作。

而且 **Skills** 在 Claude.ai、Claude Code 和 API 上都能用，一次创建，到处都能运行。

Skills 和 MCP 的关系

如果你已经了解 MCP(Model Context Protocol)，那理解 **Skills** 就更容易了。

Anthropic 用了一个很形象的比喻：

MCP 提供的是专业厨房，有工具、食材和设备；

Skills 提供的是菜谱，告诉你怎么一步步做出有价值的东西。

我这里打个更接地气的比方：

你买了一套高级厨具（MCP），但如果不知道怎么用，顶多就是炒个蛋。

但如果有本靠谱的菜谱（**Skills**），告诉你先热锅、放多少油、什么时候下料，你就能做出一桌好菜。

MCP 和 **Skills** 配合使用，才是真正的“如虎添翼”。

对比维度	MCP（连接层）	Skills（知识层）
核心作用	连接 Claude 到你的服务	教 Claude 如何有效使用服务
解决问题	“能做什么”	“该怎么做”

对比维度	MCP（连接层）	Skills（知识层）
提供内容	实时数据访问和工具调用	工作流程和最佳实践
类比	专业厨房（工具和设备）	菜谱（步骤和方法）

没有 Skills 的 MCP 集成会遇到的问题：

- 用户连上了服务，但不知道下一步该干什么
- 每次对话都从零开始，结果不一致
- 用户抱怨连接器不好用，其实真正缺的是 workflow 指导
- 支持工单不断询问“怎么用你们的集成做 X”

有了 Skills 之后：

- ☒ 预设的工作流自动激活
- ☒ 工具使用变得一致可靠
- ☒ 最佳实践内嵌在每次交互中

| 学习曲线大幅降低



说实话，我之前用一些 MCP 集成时候就有这种感觉——连上了，然后呢？

就像拿到了一堆乐高积木，但没有说明书的话，真的只能自己瞎摸索。

Skills 就是那本说明书，告诉你这些积木该怎么拼，才能搭出你想要的东西。



三种常见的 Skills 类型

Anthropic 观察到三种主要的 Skills 使用场景：

Skills 类型	适用场景	典型示例	关键技术	是否需要 MCP
文档和资产创建	生成一致的高质量输出	frontend-design skill	内嵌风格指南、模板结构、质量检查清单	❌ 不需要
工作流自动化	多步骤流程的一致执行	skill-creator skill	分步工作流、验证关卡、迭代优化循环	可选
MCP 增强	为 MCP 工具提供使用指导	sentry-code-review skill	协调多个 MCP 调用、嵌入领域专业知识	✅ 需要

三种 Claude Skills 类型对比



类型一：文档和资产创建

用来生成文档、演示文稿、应用、设计、代码等。

这类 Skills 会内嵌风格指南和品牌标准，提供模板结构，在最终确定前有质量检查清单。

重点是不需要外部工具，完全依靠 Claude 的内置能力。

这类 Skill 特别适合那些“格式要求严格，但内容需要创意”的场景。

比如你们公司的周报有固定模板，但每周内容不同，用 Skill 就能保证格式统一，同时内容又不会千篇一律。

类型二：工作流自动化

用于需要一致方法论的多步骤流程，包括跨多个 MCP 服务器的协调。

比如 Anthropic 自己做的 **skill-creator**，就是一个交互式指南，引导用户定义用例、生成元数据、编写指令、验证结果。

这样看来，我觉得其实这是最实用的类型。

你想想看，日常工作有多少事情是“步骤固定，但每次都要重复做”的？

发布流程、代码审查、客户入职……这些都可以做成 Skill。

以后只需要说一句“开始发布流程”，剩下的 Claude 自己就知道该做什么了。

类型三：MCP 增强

为 MCP 服务器提供的工具访问提供工作流指导。

Sentry 的代码审查 Skill 就是个好例子，它能自动分析和修复 GitHub Pull Request 中检测到的 bug，使用 Sentry 的错误监控数据。

这类 Skills 会按顺序协调多个 MCP 调用，嵌入领域专业知识，提供用户原本需要手动指定的上下文。

怎么开始构建 Skills

在写任何代码之前，自己先明确好 2 到 3 个具体的使用场景，这样构建 Skills 才有意义。

好的用例定义应该包括触发条件、具体步骤和预期结果。

好的用例定义示例：

用例：项目冲刺规划

触发条件：用户说"帮我规划这个冲刺"或"创建冲刺任务"

步骤：

- 1. 从 Linear 获取当前项目状态（通过 MCP）
- 2. 分析团队速度和容量
- 3. 建议任务优先级
- 4. 在 Linear 中创建带标签和估算的任务

预期结果：完整规划的冲刺，任务已创建

文件命名规则（必须严格遵守）

项目	正确格式 	错误格式 
文件夹名	notion-project-setup	Notion Project Setup notion_project_setup NotionProjectSetup
主文件名	SKILL.md (大小写敏感)	skill.md SKILL. MD Skill.md
禁止文件	不要放 README.md	 README.md

这里要特别注意一下：

首先是文件夹名必须用小写加短横线（kebab-case），不能用空格、下划线或驼峰命名。

而 **SKILL.md** 必须全大写，差一个字母都不行。

我知道这看起来有点“强迫症”，但这是系统识别 Skill 的关键，写错了 Claude 就找不到了。

YAML 元数据：最重要的部分

YAML 元数据决定了 Claude 是否会加载你的 Skills。最简单的格式只需要两个字段：

```
---

name: your-skill-name

description: 技能做什么。当用户询问[具体短语]时使用。

---
```

description 字段的黄金法则：

-  **必须包含：**技能做什么 + 什么时候使用它
-  少于 1024 个字符
-  包含用户可能说的具体任务
-  如果涉及文件类型要提到
-  不能包含 XML 标签 (< >)

好的 description 示例：

“分析 Figma 设计文件并生成开发交接文档。当用户上传 .fig 文件、询问‘设计规格’、‘组件文档’或‘设计到代码交接’时使用。”

坏的 description 示例：

“帮助处理项目” ← 太模糊，没有触发条件

我的个人经验是，写 description 的时候要站在用户角度想：

“我会用什么词来描述这个需求？”

比如做周报的 Skill，用户可能会说“写周报”、“生成周报”、“本周总结”，那你就把这些词都写进 description 里。

你写得越具体，触发就越准确。

编写有效的指令

YAML 元数据之后，就是用 Markdown 写实际的指令。

推荐的结构包括：

- 1. Skills 名称
- 2. 指令部分（分步骤说明）
- 3. 示例（常见场景和预期结果）
- 4. 故障排除（常见错误和解决方案）

指令质量对比

指令类型	坏的指令 ❌	好的指令 ✅
数据验证	“在继续之前验证数据”	“运行 <code>python scripts/validate.py --input {filename}</code> 来检查数据格式。如果验证失败，常见问题包括： • 缺少必填字段（添加到 CSV 中） • 无效的日期格式（使用 YYYY-MM-DD）“
错误处理	“处理连接错误”	“如果看到‘连接被拒绝’： 1. 验证 MCP 服务器正在运行，检查设置 > 扩展 2. 确认 API 密钥有效 3. 尝试重新连接“
资源引用	“查看文档了解详情”	“在编写查询之前，查阅 <code>references/api-patterns.md</code> 了解速率限制指导、分页模式、错误代码和处理“

三个关键原则：

- ✅ 具体可执行：提供确切的命令和参数
- ✅ 包含错误处理：预见常见问题并给出解决方案
- ✅ 使用渐进式披露：核心指令放 SKILL.md，详细文档放 references/ 目录

换句话说，写指令就像你写给实习生看的操作手册——你不能假设对方“应该知道”，而是要手把手来教。

遇到错误怎么办？常见坑在哪里？都要提前说清楚。

这样 Claude 才能真正“独立工作”，而不是每次都来打断问你。

测试和迭代

Skills 可以在不同严格程度上测试，根据你的需求选择合适的方法：

测试方法	适用场景	优点	缺点
Claude.ai 手动测试	快速迭代、个人使用	无需设置、快速反馈	不可重复、难以规模化
Claude Code 脚本测试	团队协作、重复验证	可重复、自动化	需要编写脚本
API 程序化测试	生产环境、大规模部署	系统化、可量化	设置复杂、成本高

Anthropic 推荐的方法：

先在单个挑战性任务上迭代，直到 Claude 成功，然后把成功的方法提取成 Skills。

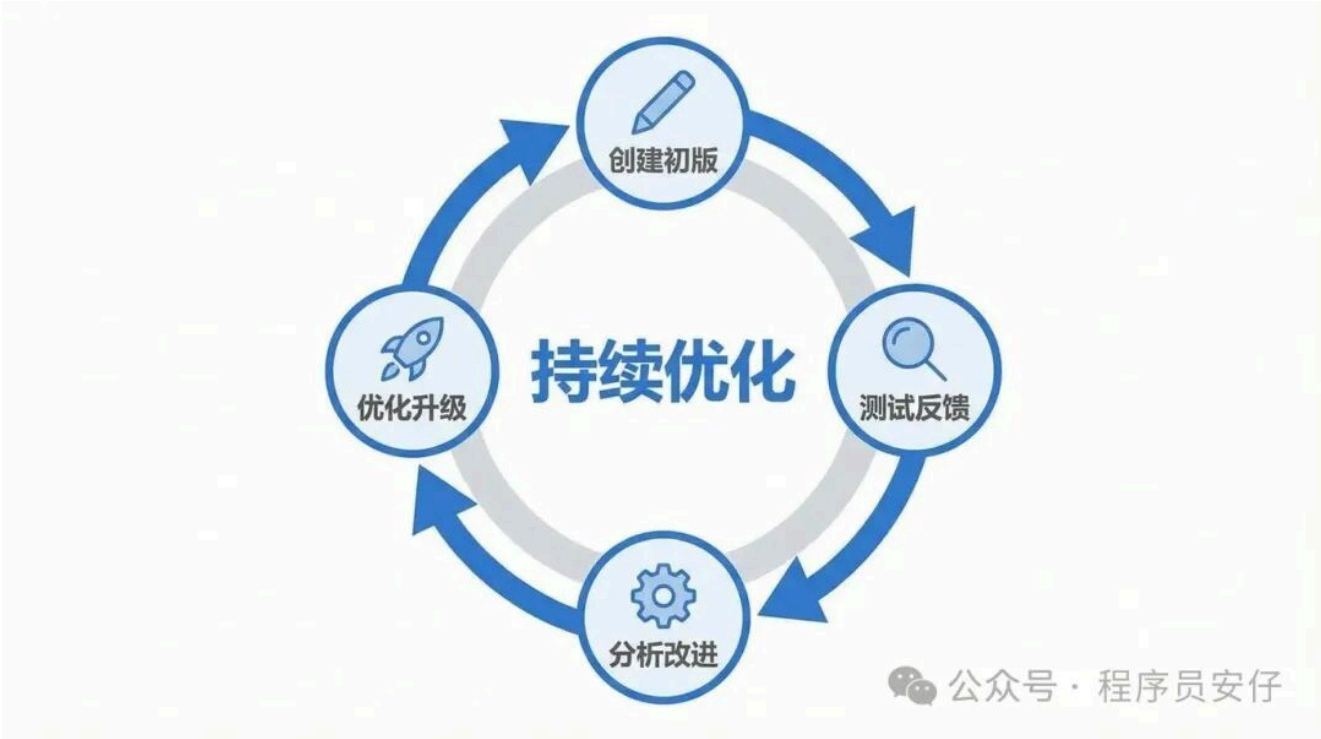
这比广泛测试更快得到反馈。

一旦有了可用的基础，再扩展到多个测试用例以获得覆盖率。

这个给我感觉就是降维打击一样，与其一开始就想着“覆盖所有场景”，不如先把一个最难的场景搞定。

就像做一款独立产品，先做个 MVP（最小可行产品），能跑通了再慢慢完善。

毕竟如果连最基本的场景都搞不定，考虑再多边缘情况也没用。



三个关键测试维度

1. 触发测试（确保 Skills 在正确的时候加载）

测试类型	测试用例示例	预期结果
✔ 应该触发	“帮我设置一个新的 ProjectHub 工作区”	Skills 自动加载
✔ 应该触发	“我需要在 ProjectHub 创建项目”	Skills 自动加载
✘ 不应该触发	“旧金山的天气怎么样？”	Skills 不加载
✘ 不应该触发	“创建一个电子表格”	Skills 不加载（除非处理表格）

2. 功能测试（验证 Skills 产生正确的输出）

- ✔ 生成有效输出
- ✔ API 调用成功
- ✔ 错误处理有效
- ✔ 边缘情况覆盖

3. 性能比较（证明 Skills 相比基线改进了结果）

对比指标	没有 Skills	有 Skills	改进
来回消息数	15 条	2 条	↓ 87%
API 失败次数	3 次	0 次	↓ 100%
Token 消耗	12,000	6,000	↓ 50%
用户干预次数	多次指导	仅需确认	显著降低

Skills 测试流程信息图



公众号 · 程序员安仔

快速构建工具：skill-creator

如果你有 MCP 服务器并知道前 2 到 3 个工作流，通常可以在 15 到 30 分钟内构建和测试一个功能性 Skills。

使用方法：

"使用 skill-creator 帮我构建一个 [你的用例] 的 Skill"

这个工具其实真的是官方直接给你了创建 Skills 的最佳实践指引了。

用了这个 Skill，它会一步步问你“想做什么”、“怎么触发”、“需要哪些步骤”，然后帮你生成一个基础版本。

你再根据实际情况微调一下，基本就能用了。

强烈建议新手从这里开始，比自己从零写要快得多。

常见问题和解决方案

问题诊断速查表

问题症状	可能原因	解决方案
Skills 不触发	description 太泛泛	添加具体触发短语和场景

问题症状	可能原因	解决方案
Skills 触发太频繁	范围定义不清	添加负面触发条件，缩小适用范围
MCP 调用失败	连接或认证问题	检查服务器状态、API 密钥、权限
指令不被遵循	指令太冗长或模糊	简化指令、加粗关键步骤、使用编号列表
无法上传	文件命名错误	确认 SKILL.md 大小写正确

详细解决方案

1. Skills 不触发

调试方法：问 Claude “你什么时候会使用 【某某 Skill】？”

它会引用 description，你就知道缺了什么。

 太泛泛

description: 帮助处理项目

 具体明确

description: 管理 Linear 项目工作流，包括冲刺规划、任务创建和状态跟踪。

当用户提到"sprint"、"Linear 任务"、"项目规划"或询问"创建工单"时使用。

2. Skills 触发太频繁

添加负面触发条件和明确范围：

description: PayFlow 电商支付处理。专门用于在线支付工作流，

不要用于一般性的财务查询（使用 finance-analysis skill）。

3. MCP 连接问题检查清单

检查项	如何验证	常见问题
✓ 服务器连接	Settings > Extensions > 【服务】	显示“已连接”状态
✓ 认证有效	检查 API 密钥和过期时间	密钥过期、权限不足
✓ 独立测试	不通过 Skills 直接调用 MCP	工具名称大小写错误
✓ 工具名称	对照 MCP 文档检查	拼写错误、版本不匹配

4. 指令不被遵循

问题类型	原因	改进方法
指令太冗长	信息过载	使用要点和编号列表，详细内容移到 references/
指令被埋没	结构不清晰	使用 ## 重要 或 ## 关键 标题
语言模糊	缺乏具体性	提供确切的命令、参数和预期输出
模型“偷懒”	缺少激励	添加“仔细执行”、“质量优先于速度”等提示

高级技巧： 对于关键验证，考虑打包一个脚本来程序化执行检查，不要只依赖语言指令。

因为代码是确定性的，但是语言解释不是。

做了那么多年程序员的我，这点我深有体会。

AI 再聪明，理解自然语言的时候还是可能有偏差。

但代码不会——要么通过，要么报错，不存在“理解错了”这回事。

所以对于那些“必须严格执行”的检查（比如数据格式验证、安全检查），最好还是写成脚本，让 Claude 调用脚本是最优解。

分发和使用

当前分发模式对比

使用场景	分发方式	适用对象	更新方式
个人使用	下载文件夹 → 压缩 zip → 上传到 Claude.ai	个人用户	手动重新上传
团队协作	放到 Claude Code 技能目录	开发团队	Git 同步
组织部署	管理员工作区范围部署	企业用户	自动更新
程序化使用	通过 API 管理和调用	开发者、应用	API 版本控制

Anthropic 的开放标准承诺：

-  Skills 已发布为**开放标准**（类似 MCP）
-  同样的 Skills 应该在 Claude 或其他 AI 平台上都能工作
-  跨平台可移植性

这里有一点要注意一下。

开放标准意味着你今天花时间做的 Skill，以后不会被“锁死”在某个平台上。

就比如你写的 Markdown/Word 文档，在哪个编辑器都能打开一样。

Anthropic 这个做法其实是值得点赞，不自己搞封闭生态，对开发者甚至是技术小白都更友好。

API 使用场景对比

使用方式	适用场景	推荐工具
Claude.ai / Claude Code	最终用户直接交互 手动测试和迭代 个人临时 workflow	网页界面或桌面应用
API	程序化使用 Skills 生产环境大规模部署 自动化管道和代理系统	<code>/v1/skills</code> 端点 Claude Agent SDK

API 关键能力：

- `/v1/skills` 端点用于列出和管理 Skills
- 通过 `container.skills` 参数将 Skills 添加到 Messages API 请求
- 通过 Claude Console 进行版本控制和管理
- 与 Claude Agent SDK 配合构建自定义代理

推荐的分发策略

在 GitHub 上托管 Skills:

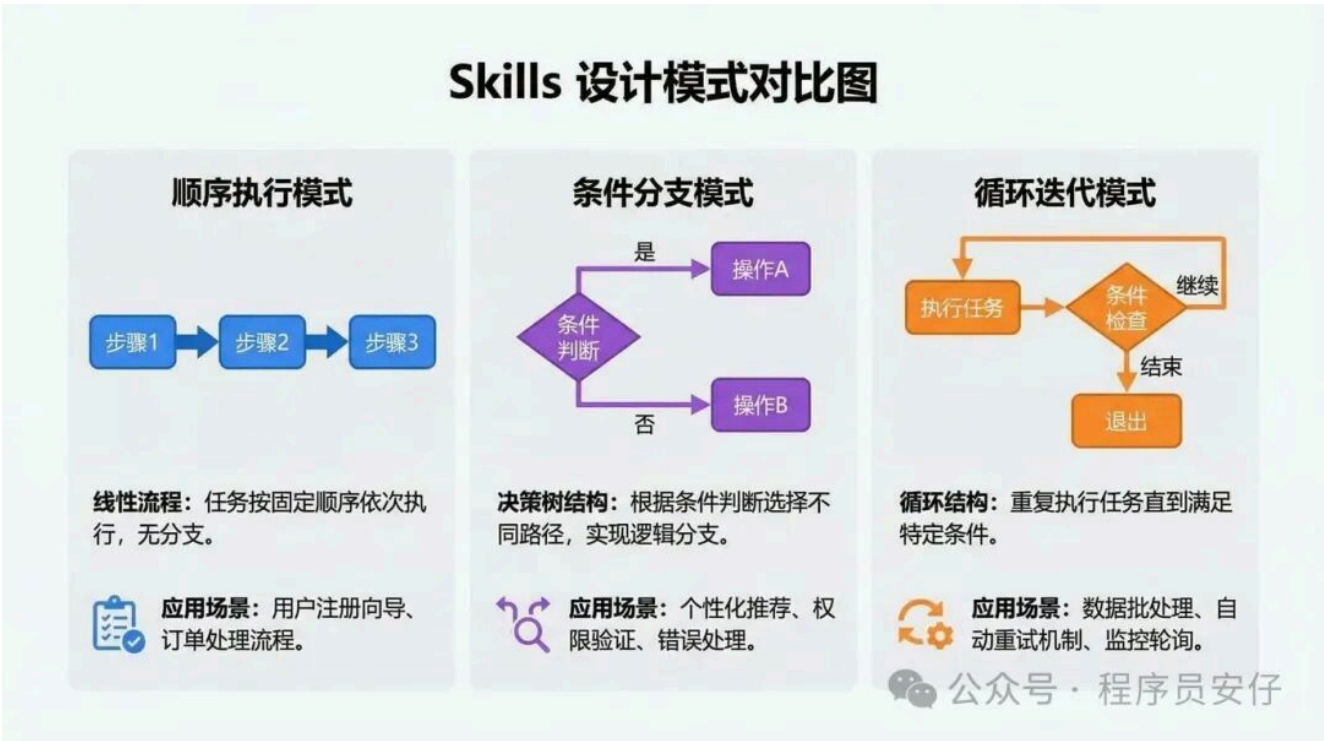
1.  创建公开仓库
2.  提供清晰的 README（给访问者看，不是放在 Skills 文件夹里）
3.  包含示例用法和截图
4.  在 MCP 文档中链接到 Skills

- 5.  解释为什么一起使用两者有价值
- 6.  提供快速入门指南

五种实用模式

基于早期采用者和内部团队的经验，Anthropic 总结了五种常见且有效的 Skills 设计模式：

模式	适用场景	典型示例	关键技术
1. 顺序 workflow 编排	需要特定顺序的多步骤流程	客户入职流程	明确步骤顺序、依赖关系、验证、回滚
2. 多 MCP 协调	跨多个服务的工作流	设计到开发交接	阶段分离、数据传递、集中式错误处理
3. 迭代优化	输出质量随迭代改进	报告生成	质量标准、验证脚本、停止条件
4. 上下文感知工具选择	根据上下文选择不同工具	智能文件存储	决策标准、备选方案、透明性
5. 领域特定智能	嵌入专业知识和合规性	支付处理	领域专业知识、合规检查、审计跟踪



模式 1：顺序 workflow 编排

使用场景：多步骤流程需要按特定顺序执行

示例：客户入职流程

步骤 1：创建账户 → 步骤 2：设置支付 → 步骤 3：创建订阅 → 步骤 4：发送欢迎邮件

关键技术：

- ☒ 明确的步骤顺序
- ☒ 步骤间的依赖关系（步骤 3 依赖步骤 1 的 customer_id）
- ☒ 每个阶段的验证
- ☒ 失败时的回滚指令

模式 2：多 MCP 协调

使用场景：工作流跨越多个服务

示例：设计到开发交接

阶段	使用的 MCP	主要操作	输出
Phase 1	Figma MCP	导出设计资产、生成规格	设计文件 + 资产清单
Phase 2	Drive MCP	创建项目文件夹、上传资产	可分享链接
Phase 3	Linear MCP	创建开发任务、附加资产链接	任务列表
Phase 4	Slack MCP	发送交接摘要到	通知确认

关键技术：

- ☒ 清晰的阶段分离
- ☒ MCP 之间的数据传递（Phase 2 的链接传给 Phase 3）
- ☒ 进入下一阶段前的验证
- ☒ 集中式错误处理

模式 3：迭代优化

使用场景：输出质量需要多次迭代改进

示例：报告生成

初稿 → 质量检查 → 识别问题 → 优化 → 再检查 → 直到达标 → 最终确定

关键技术：

-  明确的质量标准（完整性、格式一致性、数据有效性）
-  验证脚本（`scripts/check_report.py`）
-  迭代改进循环
-  知道何时停止迭代（避免无限循环）

这里稍微讲讲的是，这个模式特别适合那些“一次不一定能做好，但可以逐步改进”的任务。

就像写文章一样，初稿肯定有问题，你可以一遍遍修改。

所以你需要设定好“什么叫做好了”，不然 AI 可能会一直改下去，永远会觉得“还能更好”。

模式 4：上下文感知工具选择

使用场景：相同目标，但根据上下文使用不同工具

示例：智能文件存储决策树

文件特征	选择的存储	原因
大文件 (>10MB)	云存储 MCP	容量和带宽
协作文档	Notion/Docs MCP	实时协作
代码文件	GitHub MCP	版本控制
临时文件	本地存储	快速访问

关键技术：

-  清晰的决策标准
-  备选方案（主选项不可用时的 fallback）
-  向用户解释为什么做出这个选择（透明性）

模式 5：领域特定智能

使用场景：Skills 添加超越工具访问的专业知识

示例：带合规性的支付处理

处理前合规检查：

└─ 检查制裁名单

└─ 验证管辖权限

└─ 评估风险级别

↓

如果通过 → 处理支付 → 记录审计跟踪

如果未通过 → 标记审查 → 创建合规案例

关键技术：

- ☒ 逻辑中嵌入的领域专业知识（不只是调用工具）
- ☒ 行动前的合规检查
- ☒ 全面的文档记录
- ☒ 清晰的治理和审计跟踪

这个我感觉就是最高级的玩法了。

这里的实现就不单单只是“调用 API”了，而是把行业经验、合规要求、最佳实践都编码进 Skill 里。

这相当于是把一个资深专家的知识“固化”下来，让 AI 不仅能干活，还能“懂行”，懂“业务”的 AI 才是实用并提效的好 AI。

这对金融、医疗、法律这些高度专业化的领域特别有价值。

为什么这很重要

回过头来看，Skills 其实是让 Claude 从一个需要反复指导的助手，渐渐变成了一个真正能理解你工作方式的协作伙伴。

对不同角色的价值

角色	核心价值	具体收益
MCP 构建者	让集成更完整	• 相对于纯 MCP 的竞争优势 • 更快的用户价值实现路径 • 降低用户学习成本和支持成本
开发者和团队	标准化 AI 工作方式	• 减少重复解释 • 提高结果一致性 • 团队协作效率提升
高级用户	定制 AI workflow	• 不需要每次从头开始 • 固化个人最佳实践 • 跨项目复用经验
企业组织	规模化 AI 能力	• 工作区范围部署 • 集中管理和更新 • 确保合规性和一致性

转变对比

维度	没有 Skills 的 Claude	有 Skills 的 Claude
角色定位	需要反复指导的助手	理解你工作方式的协作伙伴
知识传递	每次对话都要重新解释	一次配置，持续使用
工作流程	手动指导每一步	自动执行标准流程
结果质量	因提示不同而变化	标准化、可预测
团队协作	每个人各自摸索	共享最佳实践

开始构建你的第一个 Skill

Anthropic 提供的资源：

-  完整的文档和最佳实践指南
-  示例 Skills 仓库（可直接复制修改）
-  skill-creator 工具（15-30 分钟快速构建）
-  活跃的开发社区（Claude Developers Discord）

现在就是最好的时机：

- 🚀 Skills 是开放标准，投入的时间不会浪费
- 🚀 生态系统正在快速发展，早期采用者有先发优势
- 🚀 构建过程比你想象的简单，但收益会持续很长时间

如果你想让 Claude 真正适应你的工作方式，从今天开始构建你的第一个 Skill 吧。

最后说一句：

Skills 这个功能，表面上看是个技术特性，但本质上是在改变我们和 AI 的协作方式。

以前是“我说一句，你做一件事”，现在是“我教你一次，你以后自己就会了”。

这种转变，可能比单纯提升模型能力更有意义。

毕竟，真正好用的工具，不是功能最多的，而是最懂你的。

对了，我最近花了半年时间，把自己用 AI 做全栈开发的经验整理成了一本书——《**人人都是 AI 程序员：TRAE+Cursor 从 0 到 1 全栈实战**》。

专门写给那些有想法、但不懂技术的朋友。

书里不讲复杂的代码，就教你像导演一样与 AI 协作，用几乎零成本的工具，在一个下午就能做出你的第一个应用。

如果你也想试试在 AI 时代当创造者，可以看看这本书：